

Beyond IID Testing. The Mismeasurement of AI?

March 27, 2025

Alan Yuille

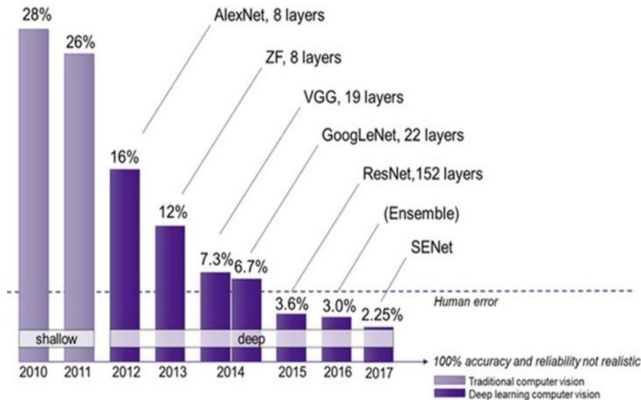
Departments of Computer Science and Cognitive Science
Johns Hopkins University

Plan of Talk

- ▶ Are we mismeasuring AI? How to best quantify AI performance?
- ▶ Computer Vision uses the standard ML paradigm which evaluates algorithms by average case performance on random set of samples. These samples are from the same source as the training data. This is called IID testing.
- ▶ This is problematic because the space of stimuli is enormous. The datasets are biased and underrepresent rare events. They do not capture the complexity of the real world.
- ▶ Current performance measures should be complemented by more advanced testing.

Part 1. The Mismeasurement of $A/$

- ▶ AI vision algorithms are very successful when measured on standard academic performance benchmarks.
- ▶ Their performance appears to be superhuman.



But they have problems.

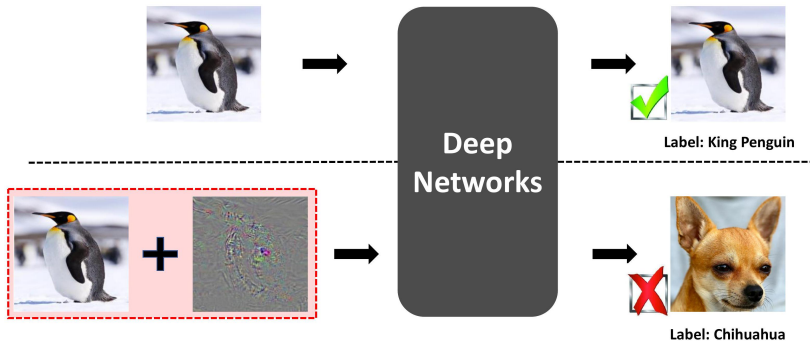
- ▶ CV algorithms fail: (i) due to image perturbations which are imperceptible to humans, (ii) due to changes to images that would not fool a human.
- ▶ CV algorithms fail in some real world conditions. E.g., Tesla's automatic vision system produces ghosts.
- ▶ Recent studies show that Deep Nets on ImageNet succeed on those images human find easy (measured by reaction time) and fail on the images humans find difficult.
- ▶ What is the problem? AI algorithms do not generalize from their training set to the real world.

Balanced Annotated Datasets and I.I.D. testing.

- ▶ All algorithms are benchmarked by average case performance on balanced annotated datasets (BADs).
- ▶ The training and testing sets in a BAD are independently and identically distributed (iid) samples from a data source domain.
- ▶ Good performance on the testing set guarantees good performance on other data from the same source.
- ▶ These assumptions seem very natural and have a solid mathematical theory supporting them (e.g., Probably Approximately Correct - Vapnik, Valiant, Smale and Poggio.)
- ▶ But they do not scale up to the complexity of the real world.

Adversarial Examples

- ▶ Deep networks are fooled by small carefully crafted perturbations.



Adversarial Examples

- ▶ Let $f(x; \theta)$ be a deep network, where x is an input image and θ are network parameters.
- ▶ To generate an adversarial example maximize the loss $E_{\text{loss}}(f(x + r; \theta), y^{\text{true}})$ wrt an Adversarial Perturbation r . Here y^{true} is the correct output.
- ▶ This r can be found by gradient ascent on $E_{\text{loss}}(f(x + r; \theta), y^{\text{true}})$ because the deep network is a differentiable function of x .
- ▶ By contrast, learning the deep network corresponds to minimizing $E_{\text{loss}}(f(x + r; \theta), y^{\text{true}})$ wrt θ (averaged over a set of samples $\{(x, y^{\text{true}})\}$)

Adversarial Examples

- ▶ Three views on Adversarial Examples: (I) They are rare stimuli that may not occur in practice. (II) They can be used by malicious agents to sabotage deep networks. (III) They suggest alternative ways of training Deep Networks by enlarging the space of stimuli.
- ▶ If (x, y^{true}) is a positive example, then it is likely that $(x + r, y^{\text{true}})$ is also a positive example, provided $r \in \mathcal{R}$ is a small perturbation. So we can enlarge the training set and the test set, by creating new images $x + r$.
- ▶ Train the network by a min max principle (A. Madry et al.). Learn θ by $\min_{\theta} \max_{r \in \mathcal{R}} E^{\text{loss}}[f(x + r, y^{\text{true}})]$ (summed over all training examples).
- ▶ Alternative perspective, retrain the deep network on the adversarial examples. Min max learning is attractive conceptually but technically challenging.

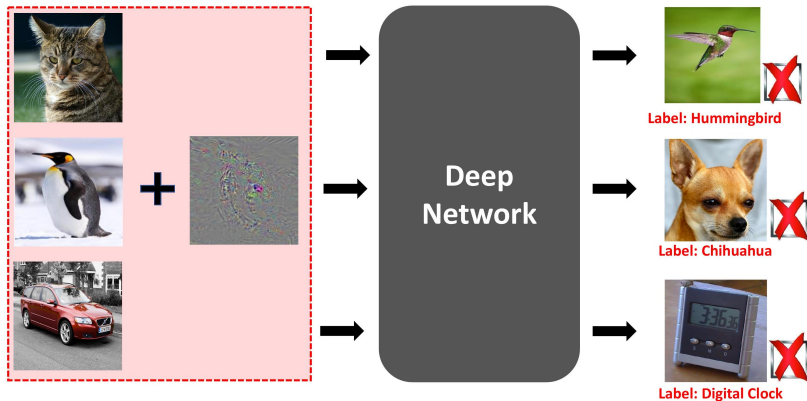
Adversarial Examples

- ▶ Is there a free lunch? It seemed likely that training on adversarial examples, in addition to standard training examples, should improve performance on both adversarial examples and standard examples.
- ▶ This was debated and the original experiments suggested there was no *free lunch*. If the deep network was trained on standard training examples *and* on adversarial examples, then its performance decreased when tested on standard examples.
- ▶ This paradox was resolved (C. Xie et al.) by noting an important detail. Training uses batch normalization. Batch normalization assumes that the data samples come from the same distribution. But the standard training examples and the adversarial examples *do not*. There is a free lunch, if batch normalization is modified so that the standard training examples and the adversarial examples are put into different batches. (Modifying the ReLu also helps a little).

Adversarial Examples

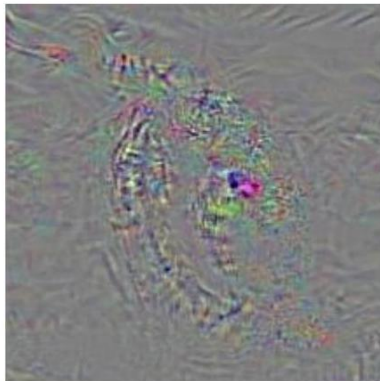
- ▶ Properties of these adversarial examples.
- ▶ They can be agnostic to the image. We can find an r which makes the deep network fail for many images.
- ▶ They can be agnostic to the deep network. An attack developed for one deep network will often succeed on another. This is a black box attack since it requires no knowledge of the weights of the deep network being attacked (unlike a white box attack).
- ▶ They can be agnostic to the task. Suggests that deep network feature vectors are common to most tasks,

Adversarial Perturbations can be Image Agnostic

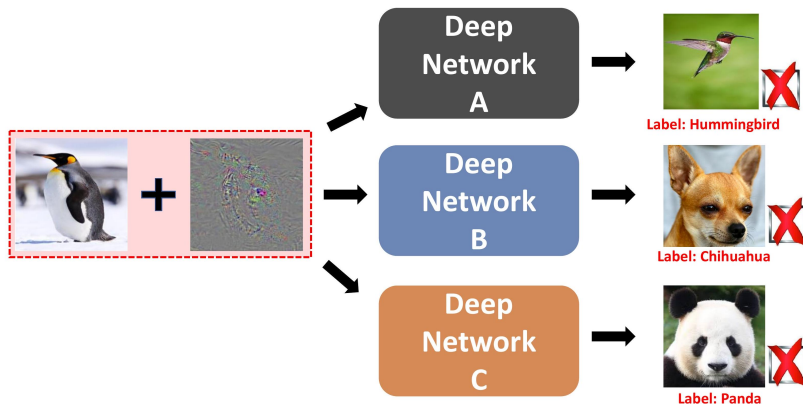


Adversarial Perturbations can be Image Agnostic

- ▶ We call such perturbations Universal Adversarial Perturbations.



Adversarial Examples can be Model Agnostic

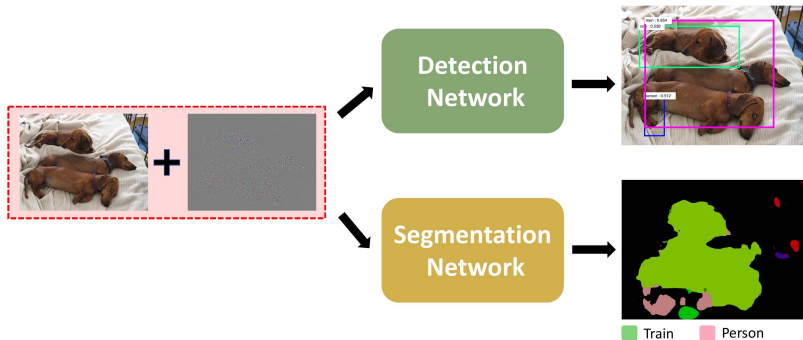


Adversarial examples exist for different tasks



Semantic segmentation. Pose estimation.

Adversarial examples transfer between different tasks

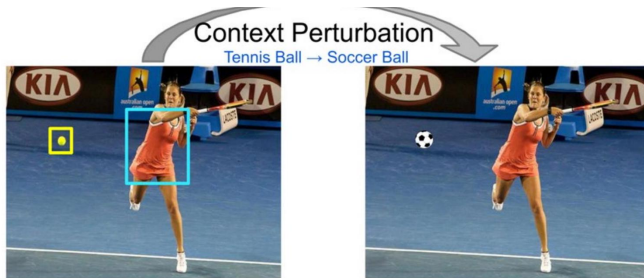


Other Types of Failures

- ▶ (I) Lack of Robustness to small changes in images that would not fool a human.
- ▶ (II) Failure to transfer/generalize to data from different domains

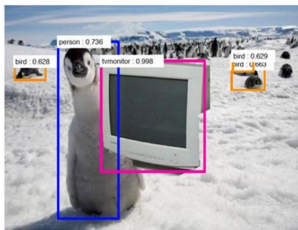
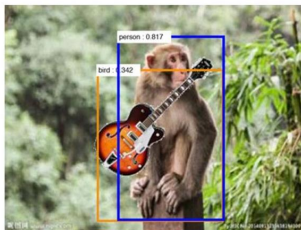
(I) Lack of Robustness to Small Changes in Images

- ▶ A recent examples from (Gupta et al. CVPR 2022).
- ▶ Changing the tennis ball to a football causes the AI algorithm to mistakenly predict the dress to be white.



(I) Lack of Robustness to Small Changes in Images

- ▶ The monkey becomes a human by photo-shopping a guitar. The guitar becomes a bird. The penguin becomes a human by photo-shopping a TV. Presumably due to over-reliance on context.



(II) Failure to transfer between domains

- ▶ Counting the number of people in images of Beijing and New York. Good performance on Beijing and New York. But algorithms trained on Beijing images work poorly on New York images, and vice versa.
- ▶ MIT Technology Article: The AI developed by Google Health can identify signs of diabetic retinopathy from an eye scan with more than 90% accuracy- at "human specialist level". principle, give a result in less than 10 minutes. The system analyzes images for telltale indicators of the condition, such as blocked or leaking blood vessels. The system worked well on datasets in Google, but mostly failed on real patients in Thailand.

(II) Failure to transfer between domains

- ▶ Edge Detection – Sowerby and South Florida dataset.



(II) Failure to transfer between domains

- ▶ Sowerby Images - English Country Scenes: significant texture in background, edges not very sharp. (figure top left: left panel)
- ▶ South Florida dataset - Mostly indoor images: texture regions removed, edges very sharp (step like). (figure top left: right panel).
- ▶ Different types of image statistics.
- ▶ But transfer is possible using Bayesian methods (Konishi et al.) $P(f_{\text{on-edge}}) P(f \mid \text{off-edge})$. The statistics $P(f \mid \text{off-edge})$ differ between datasets but can assume that $P(f \mid \text{on-edge})$ is similar. Exploit fact that most image pixels are off-edge.

Virtual Data: Controlled Datasets

- ▶ Tools like UnrealCV enable us to generate datasets which have many annotations and which test algorithms systematically.
- ▶ This enables us to systematically test algorithms.

- ▶ (i) A set of UnrealCV commands to interact with the virtual world.
- ▶ (ii) Communication between UE4 and external programs like Caffe.

-
- ▶ UnrealCV is a project to help computer vision researchers build virtual worlds using Unreal Engine 4 (UE4). It extends UE4 with a plugin by providing:

Virtual Data: Controlled Datasets.

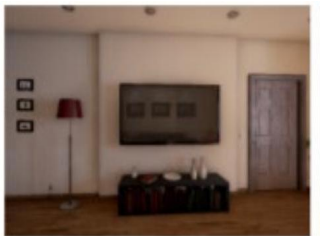
- ▶ Object detection algorithms (W. Qiu & A.L. Yuille. ECCV workshop 2016).
- ▶ E.g., Sofa detectors trained on ImageNet may not work on other data.



Elevation Azimuth	90	135	180	225	270
0	-	0.713	0.769	0.930	0.319
30	0.900	1.000	0.588	1.000	0.710
60	0.255	0.100	0.148	0.296	0.649

Table 1. The Average Precision (AP) when viewing the sofa from different viewpoints. Observe the AP varies from 0.1 to 1.0 showing the sensitivity to viewpoint. This is perhaps because the biases in the training cause Faster-RCNN to favor specific viewpoints.

- Stress-test binocular stereo. Yi Zhang et al. UnrealStereo. 3DV. 2018.



(a) Specularity



(b) No texture

Synthetic Data: Domain Transfer for Activity Recognition

- ▶ Activity Recognition is a visual task which has huge combinatorial complexity. Use synthetic data to explore this.
- ▶ Render synthetic videos of humans punching. Train state-of-the-art activity recognition methods (TSN and I3D) on these tasks using the USC101 activity dataset.
- ▶ These algorithms transfer very poorly to the synthetic data.

Model	Class Name	Top-1 accuracy	Top-5 accuracy
TSN	Punching	0.00	0.00
I3D	Punching bag	6.25	41.67
I3D	Punching person	6.25	31.25

- ▶ Why are the Deep Nets (TSN and I3D) so bad at generalizing to the synthetic data?
- ▶ (There are problems for algorithms trained on real to generalize to synthetic, but they are not usually as bad as this).

Synthetic Data: Domain Transfer for Activity Recognition

- ▶ Conjecture: TSN model trained on UCF101 (right) may have overfit to background and are unable to localize punching action. Synthetic data consists of a single boxer (left).
- ▶ Videos from this class in UCF101 are mostly boxing matches. The AI is probably exploiting the context and the clothing of boxers in matches.



Synthetic Data: Domain Transfer for Activity Recognition

- ▶ Diagnose the AI. Can TSN correctly localize the punching action?
- ▶ Class Activation Maps (CAM) are a standard technique to detect the discriminative image regions used by a CNN to identify a specific activity class.
- ▶ CAMs of punching videos from UCF101 test set. The AI is detecting: (i) ropes (context), (ii) naked torsos of boxers (clothing).



Synthetic Data: Domain Transfer for Activity Recognition

- ▶ Volleyball Spiking:



- ▶ The model is unable to localize the spiking action in time. It relies on context, i.e. the ability to recognize the scene of volleyball games.

Is the solution making bigger datasets?

- ▶ Make the datasets bigger. This arguably works for some visual tasks if the algorithms are trained on enormous amounts of data – mega data – with some human correctly.
- ▶ Example (A): Segment Anything (SAM) (informally GrabCut with learning). Conjecture: this works because it exploits two typically cues for object segmentation: (i) there are edge cues at the boundaries of objects, (ii) the texture/color inside an object differs from the texture/color in the background. Note: SAM works less well for part segmentation where these properties are not true.
- ▶ Example (B): Depth Anything. This algorithm can estimate depth up to an offset a and a scaling b – i.e., $z \mapsto a + bz$. Conjecture: This may be a statistical form of standard monocular depth cues – e.g., shape from shading and shape from texture – which exploits the shapes of standard objects.
- ▶ But it is doubtful that this strategy will work for all visual tasks. Particularly those for which the complexity of the data is combinatorial and there lack obvious cues that can generalize.

Create Out of Distribution Test Datasets.

- Create datasets where the images are generated using different causal factors. Change the weather conditions, the poses, the background context. Add occluders to the images.

Texture (40-60%)

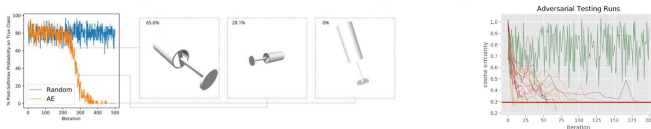


Adversarial Examiners

- ▶ More radically. **Adversarial Examiners.**
- ▶ Testing algorithms by **testing** instead of average case performance (over biased dataset).
- ▶ Design a testing procedure – an Adversarial Examiner – which searches over the stimulus space. Each test stimuli is chosen based on the response of the AI algorithm to the previous stimuli.
- ▶ This is how a professor would test a student (or vice versa). You ask a series of questions to determine how much they know about a topic. You would not ask a random series of questions.
- ▶ Software engineering protocol– (i) break the code to find its failure modes, (ii) correct the failure modes. This protocol is used for designing mature technologies – bridges, cars, airplanes – so why not use it for AI?
- ▶ This testing procedure can be learnt by reinforcement learning.

Adversarial Examiners

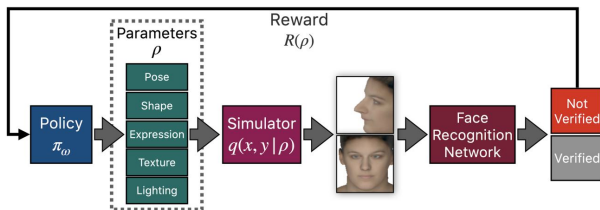
- ▶ Red/Orange - adversarial testing. Blue/green - random testing. An adversarial examiner can find weaknesses in an AI algorithm which can not be found by average case testing.



- ▶ Examples: (M. Shu et al. AAAI 2020, N. Ruiz et al. CVPR 2022).

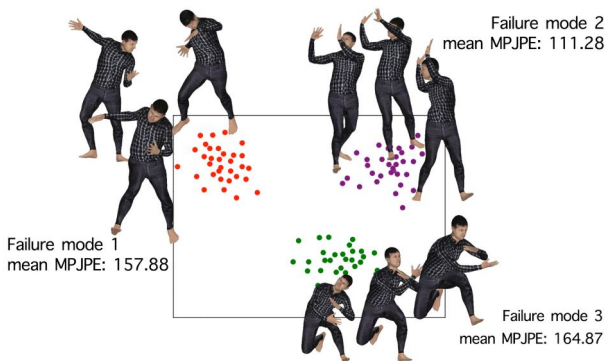
Adversarial Examiners

- Failure Models of Faces. (N. Ruiz et al. CVPR 2022).



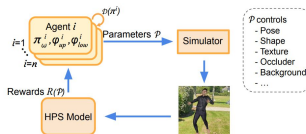
Adversarial Examiners

- Estimating 3D human pose. Pose Examiner. Q. Liu et al. 2023.



Adversarial Examiners

► Pose Examiner algorithm.



► Failure models on synthetic data and real data correspond.

